

```
In [6]: !pip install torch torchvision torchaudio scikit-learn
```

Requirement already satisfied: torch in /opt/conda/lib/python3.13/site-packages (2.13.0)

Requirement already satisfied: torchvision in /opt/conda/lib/python3.13/site-packages (0.28.0)

Requirement already satisfied: torchaudio in /opt/conda/lib/python3.13/site-packages (2.11.0)

Requirement already satisfied: scikit-learn in /opt/conda/lib/python3.13/site-packages (1.9.0)

Requirement already satisfied: filelock in /opt/conda/lib/python3.13/site-packages (from torch) (3.32.2)

Requirement already satisfied: typing-extensions>=4.10.0 in /opt/conda/lib/python3.13/site-packages (from torch) (4.16.0)

Requirement already satisfied: setuptools>=77.0.3 in /opt/conda/lib/python3.13/site-packages (from torch) (82.0.1)

Requirement already satisfied: sympy>=1.13.3 in /opt/conda/lib/python3.13/site-packages (from torch) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in /opt/conda/lib/python3.13/site-packages (from torch) (3.6.1)

Requirement already satisfied: jinja2 in /opt/conda/lib/python3.13/site-packages (from torch) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in /opt/conda/lib/python3.13/site-packages (from torch) (2026.7.0)

Requirement already satisfied: cuda-toolkit==13.0.3 in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.3.0)

Requirement already satisfied: cuda-bindings<14,>=13.0.3 in /opt/conda/lib/python3.13/site-packages (from torch) (13.3.1)

Requirement already satisfied: nvidia-cudnn-cu13==9.20.0.48 in /opt/conda/lib/python3.13/site-packages (from torch) (9.20.0.48)

Requirement already satisfied: nvidia-cusparselt-cu13==0.8.1 in /opt/conda/lib/python3.13/site-packages (from torch) (0.8.1)

Requirement already satisfied: nvidia-nccl-cu13==2.29.7 in /opt/conda/lib/python3.13/site-packages (from torch) (2.29.7)

Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in /opt/conda/lib/python3.13/site-packages (from torch) (3.4.5)

Requirement already satisfied: triton==3.7.1 in /opt/conda/lib/python3.13/site-packages (from torch) (3.7.1)

Requirement already satisfied: nvidia-cublas==13.1.1.3.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.1.1.3)

Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.88)

Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.96)

Requirement already satisfied: nvidia-cufft==12.0.0.61.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.0.0.61)

Requirement already satisfied: nvidia-nvjitlink<14,>=13.0.88 in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.3.33)

Requirement already satisfied: nvidia-cufile==1.15.1.6.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cus

```

olver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch)
(1.15.1.6)
Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.85)
Requirement already satisfied: nvidia-curand==10.4.0.35.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (10.4.0.35)
Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.0.4.66)
Requirement already satisfied: nvidia-cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.6.3.3)
Requirement already satisfied: nvidia-cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (12.6.3.3)
Requirement already satisfied: nvidia-nvtx==13.0.85.* in /opt/conda/lib/python3.13/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.3; platform_system == "Linux"->torch) (13.0.85)
Requirement already satisfied: cuda-pathfinder>=1.4.2 in /opt/conda/lib/python3.13/site-packages (from cuda-bindings<14,>=13.0.3->torch) (1.6.0)
Requirement already satisfied: numpy in /opt/conda/lib/python3.13/site-packages (from torchvision) (2.5.1)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /opt/conda/lib/python3.13/site-packages (from torchvision) (12.3.0)
Requirement already satisfied: scipy>=1.10.0 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (1.18.0)
Requirement already satisfied: joblib>=1.4.0 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: narwhals>=2.0.1 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (2.24.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in /opt/conda/lib/python3.13/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /opt/conda/lib/python3.13/site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.13/site-packages (from jinja2->torch) (3.0.3)

```

In [7]:

```

"""
Created on Thu Aug 1 19:42:57 2024

@author: vigo
"""

import os
import sys
import argparse
import time
from tqdm import trange
import numpy as np
import PIL
import numpy as np
import random
from PIL import Image, ImageDraw
from torchvision import transforms
from torch.utils.data import Dataset, DataLoader, random_split

```

```

import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
from torchvision.models import resnet18, resnet50
import matplotlib.pyplot as plt

device = torch.device('cuda' if torch.cuda.is_available() else 'mps')

print(device)

# Hyper-parameters
batch_size = 32
n_epochs = 20
learning_rate = 1e-4

# Specify the root folders for neoplastic and nonneoplastic images
neoplastic_folder = './PCData/Neoplasia'
nonneoplastic_folder = './PCData/Non-neoplasia'

nonneoplastic_data = [(os.path.join(nonneoplastic_folder, file), 0) for file in os.listdir(nonneoplastic_folder)]
neoplastic_data = [(os.path.join(neoplastic_folder, file), 1) for file in os.listdir(neoplastic_folder)]
data = nonneoplastic_data + nonneoplastic_data

# Create the train dataset using train_data.txt
train_data = []
train_ids = set() # Define train_ids before adding elements to it
# Assuming train_data.txt contains data in the format: id \t ccenum \t class_type
with open('train_data.txt', 'r') as file:
    for line in file:
        # Assuming train_data.txt contains data in the format: id \t ccenum \t class_type
        sdkid = line.strip().split(',')[0]
        train_ids.add(sdkid)

# Iterate through your neoplastic and nonneoplastic data
for filepath, label in data:
    filename = os.path.basename(filepath)
    sdkid_train = filename.split('_')[0] # Assuming sdkid is the prefix before _
    #print(sdkid_train)
    if sdkid_train in train_ids:
        train_data.append((filepath, label))

# Create the valid dataset using valid_data.txt
valid_data = []
valid_ids = set() # Define train_ids before adding elements to it
# Assuming train_data.txt contains data in the format: id \t ccenum \t class_type
with open('valid_data.txt', 'r') as file:
    for line in file:
        # Assuming train_data.txt contains data in the format: id \t ccenum \t class_type
        sdkid = line.strip().split(',')[0]
        valid_ids.add(sdkid)

```

```

# Iterate through your neoplastic and nonneoplastic data
for filepath, label in data:
    filename = os.path.basename(filepath)
    sdkid_val = filename.split('_')[0] # Assuming sdkid is the prefix before '_'
    #print(sdkid_data)
    if sdkid_val in valid_ids:
        valid_data.append((filepath, label))

# Create the test dataset using train_data.txt
test_data = []
test_ids = set() # Define train_ids before adding elements to it
# Assuming train_data.txt contains data in the format: id \t ccenum \t class_type
with open('test_data.txt', 'r') as file:
    for line in file:
        # Assuming train_data.txt contains data in the format: id \t ccenum \t c
        sdkid = line.strip().split(',')[0]
        test_ids.add(sdkid)

# Iterate through your neoplastic and nonneoplastic data
for filepath, label in data:
    filename = os.path.basename(filepath)
    sdkid_test = filename.split('_')[0] # Assuming sdkid is the prefix before '_'
    #print(sdkid_data)
    if sdkid_test in test_ids:
        test_data.append((filepath, label))

class CustomDataset(Dataset):
    def __init__(self, data, transform=None):
        self.data = data
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        img_path, label = self.data[idx]
        image = Image.open(img_path).convert('RGB')

        if self.transform:
            image = self.transform(image)

        return image, label, os.path.basename(img_path)

# Define a variety of augmentations without resizing
transform = transforms.Compose([
    transforms.Resize((224, 224)), # Resize images to 224x224
    #transforms.RandomHorizontalFlip(),
    #transforms.RandomVerticalFlip(),
    #transforms.RandomAffine(degrees=15, translate=(0.1, 0.1), scale=(0.9, 1.1)),
    #transforms.GaussianBlur(kernel_size=3),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)), # Normalize images
])

# Create the custom dataset instances for training, validation, and testing
train_dataset = CustomDataset(data=train_data, transform=transform)

```

```

valid_dataset = CustomDataset(data=valid_data, transform=transform)
test_dataset = CustomDataset(data=test_data, transform=transform)

# Create DataLoader for training, validation, and testing
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

```

cuda

```

In [8]: def unnormalize(img):
        img = img * 0.5 + 0.5 # reverse normalization
        return img

# select 16 random indices for plotting
indices = random.sample(range(len(train_dataset)), 16)

plt.figure(figsize=(10, 10))

for i, idx in enumerate(indices):
    image, label, image_name = train_dataset[idx]

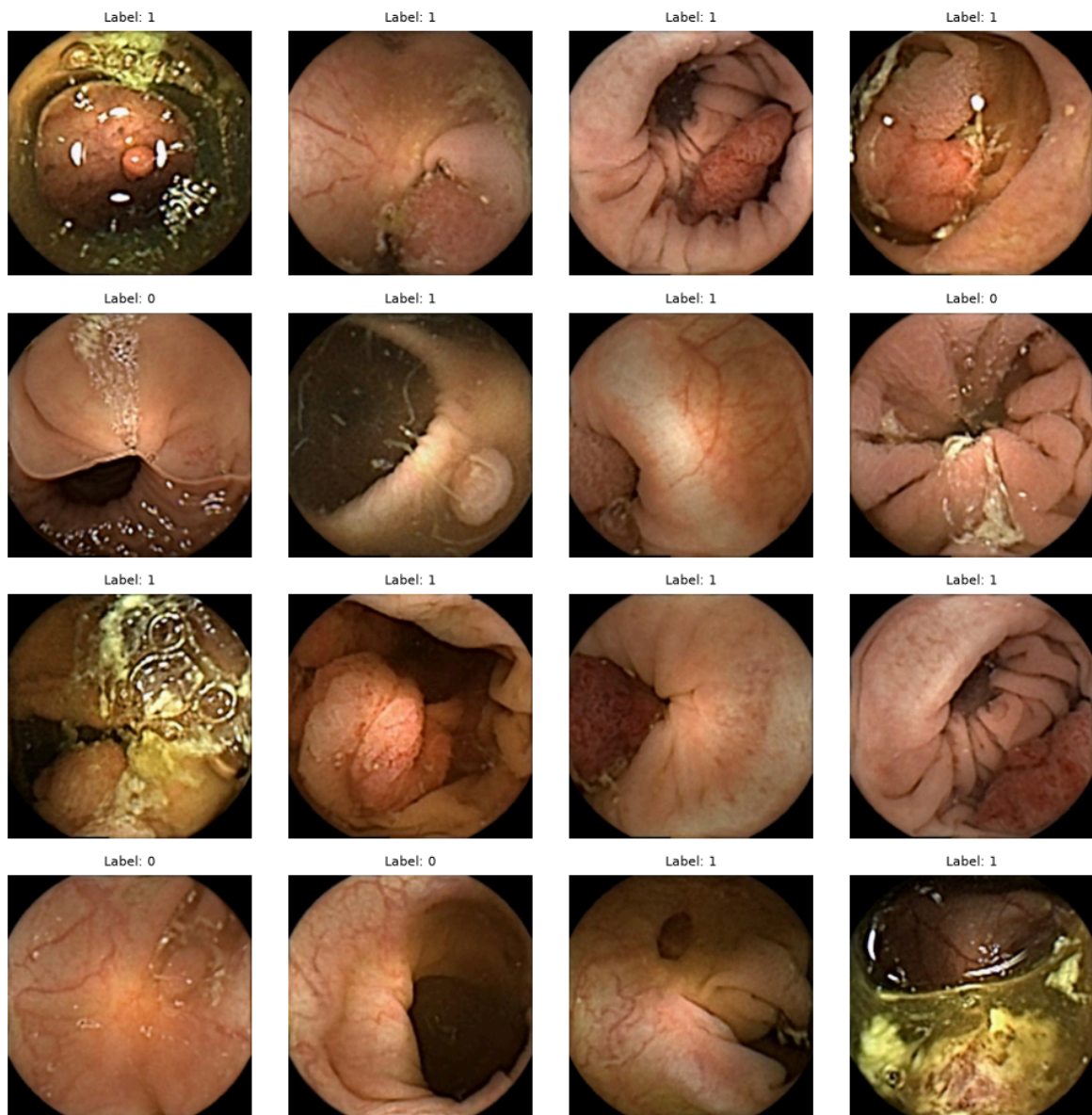
    if isinstance(image, torch.Tensor):
        image = image.numpy()

    if image.shape[0] == 3: # RGB
        image = np.transpose(image, (1, 2, 0))
        image = unnormalize(image)
    elif image.shape[0] == 1: # Grayscale
        image = image.squeeze(0)
        image = unnormalize(image)

    plt.subplot(4, 4, i + 1)
    plt.imshow(image, cmap='gray' if image.ndim == 2 else None)
    plt.title(f"Label: {label}", fontsize=8)
    plt.axis('off')

plt.tight_layout()
plt.show()

```



```
In [9]: # Initialize the model, criterion, optimizer
pc_model = resnet18(pretrained=True).to(device) # Single output for binary clas
pc_model.fc = nn.Linear(pc_model.fc.in_features, 1)
pc_model.to(device)
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(pc_model.parameters(), lr=learning_rate, betas=(0.9
```

/opt/conda/lib/python3.13/site-packages/torchvision/models/_utils.py:207: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

warnings.warn(
/opt/conda/lib/python3.13/site-packages/torchvision/models/_utils.py:222: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet18\_Weights.IMAGENET1K\_V1`. You can also use `weights=ResNet18\_Weights.DEFAULT` to get the most up-to-date weights.

warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /home/uccloud/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth

```
0%|          | 0.00/44.7M [00:00<?, ?B/s]
10%|█         | 4.38M/44.7M [00:00<00:00, 45.7MB/s]
59%|███████   | 26.5M/44.7M [00:00<00:00, 155MB/s]
```

100% |██████████| 44.7M/44.7M [00:00<00:00, 165MB/s]

```

In [10]: def train(model, device, dataloader, criterion, optimizer):
    model.train()
    train_correct = 0
    n_total_steps = len(dataloader.sampler)
    for images, targets, image_names in dataloader:
        images, targets = images.to(device), targets.to(device).float().view(-1,

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, targets)
        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        predictions = (torch.sigmoid(outputs) > 0.65).float()
        train_correct += (predictions == targets).sum().item()

    # Calculate training accuracy
    train_acc = train_correct / n_total_steps * 100
    return train_acc

def val(model, device, dataloader, criterion):
    n_total_samples = 0
    val_loss = 0
    val_correct = 0
    model.eval()
    with torch.no_grad():
        for images, targets, image_names in dataloader:
            images, targets = images.to(device), targets.to(device).float().view

            # Forward pass
            outputs = model(images)
            loss = criterion(outputs, targets)
            val_loss += loss.item()

            predictions = (torch.sigmoid(outputs) > 0.65).float()
            val_correct += (predictions == targets).sum().item()
            n_total_samples += targets.size(0)

    val_acc = val_correct / n_total_samples * 100
    avg_val_loss = val_loss / len(dataloader)
    return avg_val_loss, val_acc

for e in range(n_epochs):
    train_acc = train(pc_model, device, train_loader, criterion, optimizer)
    val_loss, val_acc = val(pc_model, device, val_loader, criterion)
    print(f'Epoch [{e+1}/{n_epochs}], Train Accuracy: {train_acc:.2f}%, Val Accu

test_loss, test_acc = val(pc_model, device, test_loader, criterion)
print(f'Test Accuracy: {test_acc:.2f}%')

```

```

Epoch [1/20], Train Accuracy: 66.81%, Val Accuracy: 86.15%, Val Loss: 0.2402
Epoch [2/20], Train Accuracy: 97.61%, Val Accuracy: 94.60%, Val Loss: 0.1502
Epoch [3/20], Train Accuracy: 99.20%, Val Accuracy: 92.72%, Val Loss: 0.1899
Epoch [4/20], Train Accuracy: 99.71%, Val Accuracy: 92.72%, Val Loss: 0.2364

```

```
Epoch [5/20], Train Accuracy: 98.48%, Val Accuracy: 93.19%, Val Loss: 0.1997
Epoch [6/20], Train Accuracy: 99.57%, Val Accuracy: 92.25%, Val Loss: 0.2074
Epoch [7/20], Train Accuracy: 98.19%, Val Accuracy: 94.13%, Val Loss: 0.1992
Epoch [8/20], Train Accuracy: 99.35%, Val Accuracy: 95.54%, Val Loss: 0.1312
Epoch [9/20], Train Accuracy: 99.86%, Val Accuracy: 93.66%, Val Loss: 0.2145
Epoch [10/20], Train Accuracy: 99.57%, Val Accuracy: 96.01%, Val Loss: 0.1482
Epoch [11/20], Train Accuracy: 99.93%, Val Accuracy: 95.31%, Val Loss: 0.1590
Epoch [12/20], Train Accuracy: 100.00%, Val Accuracy: 94.60%, Val Loss: 0.1845
Epoch [13/20], Train Accuracy: 100.00%, Val Accuracy: 95.07%, Val Loss: 0.1717
Epoch [14/20], Train Accuracy: 100.00%, Val Accuracy: 95.31%, Val Loss: 0.1789
Epoch [15/20], Train Accuracy: 100.00%, Val Accuracy: 94.84%, Val Loss: 0.1654
Epoch [16/20], Train Accuracy: 100.00%, Val Accuracy: 94.37%, Val Loss: 0.1827
Epoch [17/20], Train Accuracy: 99.86%, Val Accuracy: 92.02%, Val Loss: 0.2473
Epoch [18/20], Train Accuracy: 96.09%, Val Accuracy: 92.96%, Val Loss: 0.2742
Epoch [19/20], Train Accuracy: 97.54%, Val Accuracy: 92.49%, Val Loss: 0.2345
Epoch [20/20], Train Accuracy: 97.97%, Val Accuracy: 93.66%, Val Loss: 0.1948
Test Accuracy: 96.31%
```

In []: